

Examen Parcial de IA

(3 de abril de 2019)

grupo 10

Duración: 55 min

1. (4 puntos) Los organizadores de la pasarela Gaudí nos piden un sistema inteligente que ayude a planificar los desfiles de la próxima edición. El sistema ha de organizar diez desfiles para diez diseñadores, dos por día en un total de cinco días, sabiendo que algunos diseñadores se niegan a desfilan el mismo día que algún otro. Se han de asignar modelos a los desfiles (m por desfile) sin que la misma persona desfile más de tres veces en total durante la semana. Cada diseñador tiene también unas restricciones de altura y peso para sus modelos. Los organizadores de la pasarela han de pagar a los modelos y cada uno tiene su tarifa por participar en un desfile. Obviamente, los organizadores quieren pagar lo menos posible. Tras un análisis inicial del problema un compañero de nuestra empresa nos plantea dos estrategias distintas para resolverlo:

- a) Usar el algoritmo de A^* . El estado es una asignación parcial de diseñadores y modelos a desfiles. Se definen dos operadores: **añadir_diseñador** a un desfile concreto, que comprueba que el diseñador no se haya añadido antes y que en caso de ser el segundo diseñador asignado del día, no haya incompatibilidad entre ambos diseñadores, el coste de este operador es siempre 1; y **añadir_modelo** a un desfile concreto, que comprueba que el desfile tenga ya diseñador asignado, que no haya más de m modelos en el desfile y que se cumplan las restricciones del diseñador, el coste de este operador es la tarifa del modelo asignado. Como función heurística usamos el número de diseñadores que falta incluir en los desfiles más el número de modelos que falta incluir en los desfiles multiplicado por la tarifa del modelo más caro.

El algoritmo A^* está pensado para encontrar la solución óptima y no solo para maximizar o minimizar según uno o más criterios. Utilizar un algoritmo con un coste tan elevado en un problema en el que no se requiere encontrar el óptimo es una mala decisión, ya que introducimos una complejidad algorítmica innecesaria.

El estado está plantado como la asignación parcial de diseñadores y modelos a desfiles, lo cual es correcto si se parte del estado vacío (sin asignaciones), de forma que los operadores vayan construyendo la solución.

El operador **añadir_diseñador** es correcto, permite avanzar hacia una solución asignando un diseñador a un desfile evitando asignar más de un desfile por diseñador y evitando incompatibilidades entre diseñadores el mismo día. Faltaría comprobar que el desfile no tenga diseñador asignado. El factor de ramificación es $O(\text{diseñadores} \cdot \text{desfiles})$, siendo *diseñadores* el número de diseñadores y *desfiles* el número de desfiles (en este problema ambos son 10, así que tenemos siempre un factor de ramificación de 100).

El operador **añadir_modelo** también es correcto, permite avanzar hacia una solución asignando un modelo a un desfile que ya tenga diseñador asignado y que cumpla las restricciones del diseñador a sus modelos, y mirando también que no haya más de m modelos en un desfile. Faltaría comprobar en este segundo operador que un modelo no se asigne a más de 3 desfiles, que no se le asigne a dos desfiles el mismo día (e incluso que no se asigne el modelo dos veces al mismo desfile). El factor de ramificación es $O(\text{modelos} \cdot \text{desfiles})$, siendo *modelos* el número de modelos y *desfiles* el número de desfiles (en este problema, al tener solo 10 desfiles, podríamos considerarlo lineal respecto a los modelos: $O(\text{modelos})$).

La función $g(n)$ va sumando 1 por cada diseñador añadido y tf_i (tarifa del modelo i) por cada modelo añadido. La función $h(n)$ incluye el número de diseñadores que falta por colocar más una estimación de lo que hay que pagar por los modelos que faltan por colocar (número de modelos que falta incluir en los desfiles para llegar a m multiplicado por la tarifa del modelo más caro). Así que podemos decir que g y h son funciones que están alineadas. El primer problema es que la función f ($f(n) = g(n) + h(n)$) incluye elementos

que están en diferentes unidades (número de diseñadores -entre 0 y 10-, tarifa a pagar -posiblemente varios miles de euros-) con rangos de valores muy dispares y por ello sería bueno añadir alguna ponderación, tanto en g como en h . El segundo problema es que g tiene el número de diseñadores asignados y h tiene el número de diseñadores por asignar, y por lo tanto en f el número de diseñadores es constante y no tiene influencia en la elección de los estados sucesores (y en h , por falta de ponderación, el algoritmo tenderá a añadir modelos antes que diseñadores). Pero el problema más importante es que h no es admisible al ser un estimador pesimista, ya que utiliza la tarifa del modelo más caro como estimación del coste de los modelos por añadir a la solución, y seguramente la asignación final tendrá un coste menor. Se debería escoger la tarifa menor de los modelos en la estimación de h .

- b) Usar un algoritmo de satisfacción de restricciones. El grafo de restricciones tendría tres variables por cada uno de los modelos que representan los tres posibles desfiles que se le pueden asignar a cada modelo ($desfile_1modelo_x$, $desfile_2modelo_x$, $desfile_3modelo_x$) y una variable por cada uno de los 10 diseñadores ($diseñador_y$). El dominio de cada una de esas variables sería el identificador del desfile a asignar (un número entero entre el 1 y el 10, asumiendo que 1 y 2 son los desfiles del primer día, 3 y 4 son los desfiles del segundo día, y así sucesivamente). Como restricciones binarias habría: desigualdad entre todo par de variables de un mismo modelo x ($desfile_zmodelo_x$ y $desfile_wmodelo_x$) de modo que tengan asignado un desfile diferente; desigualdad entre todo par de variables $diseñador_z$ y $diseñador_w$ (cada diseñador ha de tener asignado un desfile diferente), y para cada modelo x que no cumple las restricciones de altura y peso de un diseñador y , establecer también una restricción binaria de desigualdad entre la variable $diseñador_y$ y las tres variables del modelo x (el modelo x no puede coincidir en ninguno de sus desfiles asignados con el diseñador y).

Este planteamiento no es correcto, ya que un algoritmo de satisfacción de restricciones no está pensado para maximizar o minimizar según unos criterios, sino solo para obtener soluciones que cumplan unas restricciones.

La representación está bien planteada, las variables representan la asignación de diseñadores y modelos a desfiles y nos permiten representar una solución en el momento en el que reducimos los dominios de todas esas variables a un solo valor. Las variables $diseñador_w$ nos permiten representar el único desfile asignado a cada diseñador, y las tres variables $desfile_zmodelo_x$ para cada modelo permiten representar los tres desfiles que como máximo se pueden asignar a un modelo, pero en éstas falta añadir a su dominio el valor vacío. La representación no es del todo eficiente si tenemos un número muy grande de modelos, ya que habrá varias de esas variables asignadas al valor vacío.

La restricción binaria de desigualdad entre variables $diseñador_w$ es correcta, fuerza que cada diseñador esté en un desfile diferente. La restricción de que cada desfile solo puede tener un diseñador está implícita en la representación, al tener 10 desfiles y 10 diseñadores a los que se ha de asignar un desfile diferente. Pero faltaría añadir una restricción binaria *día_diferente* entre todo par de variables de diseñadores que no quieran desfilan en el mismo día.

La restricción de que un modelo pueda desfilan un máximo de 3 veces en la semana es implícita en la representación propuesta. Las restricciones binarias de desigualdad entre todo par de variables de un mismo modelo x ($desfile_zmodelo_x$ y $desfile_wmodelo_x$) no es del todo correcta, ya que no es suficiente que a un modelo se le asignen un máximo de tres desfiles diferentes, sino que estos han de ser en tres días diferentes (la restricción debería ser *día_diferente*).

Faltaría también una restricción global que afecte solo a las variables $desfile_zmodelo_x$ para verificar que haya exactamente m modelos asignados a cada desfile.

Comenta cada una de las posibilidades indicando si resuelven o no el problema, qué errores te

parece que tiene cada solución y cómo se podrían corregir, y qué ventajas e inconvenientes tienen cada una de ellas. Justifica la respuesta.

2. (6 puntos) Una empresa de transporte marítimo desea decidir la configuración de la carga de su próximo barco. La empresa recibe un conjunto de peticiones de envío de entre las que escoger. Cada petición tiene asociado un precio de transporte.

Cada petición va dentro de un tipo contenedor que está fijado por el tamaño, el peso del envío y las características de la carga. Existen solamente K tipos de contenedor. Los contenedores son propiedad de la empresa y su uso tiene un coste asociado que depende del tipo de contenedor.

Existen diferentes restricciones para la carga: la suma total de pesos de los contenedores no ha de sobrepasar la capacidad de carga del barco P_{max} , ni ha de ser inferior a cierto valor P_{min} . Cada contenedor tiene un peso asociado que depende de la carga que contiene. Las posibilidades de colocar los contenedores en el barco también imponen que haya un mínimo de C_{min} contenedores y un máximo de C_{max} contenedores de cada tipo.

El objetivo es encontrar la combinación de peticiones que maximicen el precio de transporte y minimicen el coste y que esté dentro de las restricciones impuestas.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística, ...). El objetivo es comentar la solución que se propone respecto a si es correcta, es eficiente, o es mejor o peor respecto a otras alternativas posibles. Justifica tu respuesta.

- a) Se plantea solucionarlo mediante Hill-climbing utilizando como solución inicial el barco vacío y como operadores añadir y quitar contenedores del barco. Queremos usar como función de evaluación de las soluciones lo siguiente:

$$h(n) = \sum_{i=0}^{Ncont} Precio_i \times \sum_{i=0}^{Ncont} Coste_i$$

Donde $Ncont$ es el número de contenedores que hay en una solución, $Precio_i$ es el precio de transporte del contenedor i de la solución y $Coste_i$ es el coste de un contenedor i de la solución.

Plantear como solución un Hill Climbing para este problema es adecuado a priori, ya que nos piden encontrar una solución maximizando o minimizando una serie de criterios sin necesidad de obtener el óptimo.

La solución inicial planteada no es solución (la solución vacía no cumple ninguno de los límites C_{min} , C_{max} , P_{min} o P_{max} al menos que alguno de ellos sea cero). El coste de generación es cero pero su calidad es mala, dejando todo el trabajo de construir una primera solución a la búsqueda. De tener una buena combinación de operadores y heurístico podría compensarse este problema, pero no es el caso. Además para este problema no es difícil construir soluciones iniciales mejores (que cumplan C_{min} y P_{min}) con costes lineales, por lo que no es recomendable empezar con la solución vacía.

Los operadores tienen el potencial de cubrir todo el espacio de búsqueda y ambos tienen un factor de ramificación ($O(peticiones)$) lineal respecto al número de peticiones (dado que hay exactamente un contenedor por petición). Pero ninguno de los operadores comprueba las restricciones sobre los límites (C_{min} , C_{max} , P_{min} o P_{max}), por lo que pueden sacarnos fácilmente del espacio de soluciones si el heurístico no lo controla.

La función heurística multiplica un criterio que queremos maximizar (*precio*) por otro que queremos minimizar (*coste*), y por ello no nos sirve ni maximizarla ni minimizarla. Lo mejor sería plantear una resta entre el precio y el coste, ya que así estaríamos calculando el beneficio obtenido por contenedor. El heurístico debería también añadir una penalización de las no-soluciones que sea proporcional a lo lejos o cerca que esté la solución parcial de los límites que no se cumplan.

- b) Se plantea solucionarlo mediante Hill-climbing utilizando como solución inicial escoger al azar C_{min} contenedores de cada tipo. Como operador utilizamos intercambiar un contenedor del barco por otro que no esté en él. Queremos usar como función de evaluación de las soluciones lo siguiente:

$$h(n) = \sum_{i=0}^{N_{cont}} \frac{Precio_i}{Coste_i}$$

Como se ha dicho en el apartado anterior, a priori Hill Climbing es adecuado para este tipo de problema.

La solución inicial ahora cumple una de las restricciones (C_{min} contenedores de cada tipo) y tiene un coste de generación de $O(K \cdot C_{min})$ pero es probable que en algunos casos no cumpla la restricción de peso mínimo (P_{min}) y podría por lo tanto no ser solución.

En este caso el único operador no cubre todo el espacio de soluciones, ya que no puede modificar el número de contenedores de la solución inicial. Su factor de ramificación es ($O(peticiones^2)$). Además el operador no comprueba las restricciones sobre los límites (C_{min} , C_{max} , P_{min} , P_{max}), por lo que puede sacarnos fácilmente del espacio de soluciones si el heurístico no lo controla.

En el heurístico se combinan los dos criterios del enunciado, el precio (a maximizar) y el coste (a minimizar). Al colocar el precio en el numerador y el coste en el denominador, en este caso este heurístico que se propone se ha de maximizar. El problema es que este cociente puede darnos valores similares con precios bajos y costes bajos o con precios altos y costes altos, y es difícil de controlar la influencia de cada uno de los criterios (sería mejor una resta de precio menos coste, ya que además, en este caso, calcularía el beneficio obtenido por contenedor). El heurístico debería también añadir una penalización de las no-soluciones que sea proporcional a lo lejos o cerca que esté la solución parcial de los límites que no se cumplan.

- c) Se plantea utilizar algoritmos genéticos donde la representación de la solución es una tira de bits con tantos bits como peticiones haya, donde cada bit significa si la petición está o no en el barco. Para generar la población inicial usamos la misma técnica que en el apartado anterior. Usamos los operadores habituales de cruce y mutación y como función de evaluación usamos la del apartado anterior pero haciendo que valga cero si la solución incumple las restricciones del problema.

Plantear como solución un algoritmo genético para este problema es adecuado a priori, ya que nos piden encontrar una solución maximizando o minimizando una serie de criterios sin necesidad de obtener el óptimo.

La representación escogida permite representar cualquier solución del problema, pero también permite representar muchas no-soluciones (las que no cumplen los límites C_{min} , C_{max} , P_{min} o P_{max}).

La función generadora de la población inicial escogida (la del apartado anterior) solo genera un individuo, deberíamos llamarla tantas veces como individuos queramos en la población inicial. Al tener un factor aleatorio nos dará individuos ligeramente diferentes, pero de variabilidad limitada, ya que todos ellos tendrán el número mínimo de contenedores necesarios para cumplir las restricciones C_{min} . Además, al igual que en el apartado anterior, es probable que varias de esas soluciones iniciales no sean solución al incumplir la restricción sobre P_{min} . El coste de generación será $O(N \cdot K \cdot C_{min})$, siendo N el tamaño escogido de la población inicial.

Los operadores de cruce y mutación estandar escogidos, dada la posibilidad de que la representación codifique no-soluciones, pueden sacarnos del espacio de búsqueda fácilmente. Y el heurístico no lo controla adecuadamente, como veremos a continuación.

La función de evaluación propuesta (una modificación de la del apartado anterior) es a priori adecuada en un algoritmo genético, ya que se ha de maximizar, por lo que puede funcionar

como función de fitness (valores mayores indican un mejor fitness de la solución) aunque funcionaría aun mejor la resta entre el precio y el coste (así las soluciones de mayor fitness serían las que mayor beneficio neto aportan). En este caso se penalizan las no-soluciones pero siempre haciendo que la función valga 0, y así no da información al algoritmo de cuan cerca o lejos está una solución parcial de ser una solución total. Sería mejor tener una penalización proporcional a cuan cerca o lejos se está de los límites, tal y como se ha comentado en los apartados anteriores.